Enter Hibernate
Hibernate Architecture
CRUD operations in Hibernate

1. What is Hibernate?
Refer to readme

Complete solution to manage automatic persistence in DB in Java.
ORM tool
JPA implementor
JPA : Java Persistence API --- Java EE / Jakarata EE specs
packge - javax.persistence | jakarta.persistence
Hibernate : most popular JPA implementor

(DB Journey in Java ---1. JDBC 2. Hibernate (native hibernate) 3. JPA 4. Spring Data JPA)

Hibernate :auto persistence provider

Other persistence providers : iBatis,Kodo, EclipseLink,TopLink,JDO

Spring Boot frmwork : default persistence provider = Hibernate

Open source framework : founded by Gavin King

Intermediate layer between Java app(standalone desk top based / web app) n DB

Why Hibernate ? (refer to readme)
1. open source n light weight
2. supports cache (L1 , L2 , query cache) for faster performance
3. auto table creation.
4. simplifies join queries
5. Supports 100 % DB independence.
Hibernate will translate DB independent queries(HQL - Hibernate Query Language /JPQL - Java Persistence Query Language) to DB specific SQL syntax , using DB Dialect property.

6. Hibernate developer doesn't have
 to go to DB level , DB ,table ,cols , rows
 sql
 set up the db conn , prepare stmts (st/pst/cst)
 exec queries : process RST : convert it into pojo / collection of POJOs
All of above will be automated by Hibernate

7. JDBC : fixed db conn.(new separate conn per call to DriverManager.getConnection)
Hibernate creates :internal connection pool => collection of DB connections
when : hibernate framework booting time
at the time of creation of singleton SessionFactory(SF)
at the time confgure() -- hibernate.cfg.xml(location : by default run time class path) is parsed :
DB config details -- drvr class , db url , user name , pwd
hibernate.connection.pool_size= 10 (max size)

In DAO layer : When you invoke , open session n begin tx : db conn is pooled out -- wrapped in Session instance n reted to caller.
try

CRUD work (save/get/JPQL/update/delete...)
end of try --commit
catch --RunTimeExc --rollback
finally : session .close ---pooled out db cn simply rets to the pool : so that the same conn can be REUSED for some other request.

8.Solves the important issue of Impedance mismatch in DBMS
Object world (java objs in heap , inheritance , association , polymorphism) ----- RDBMS (table , row cols ,E-R,FKs,join tables...)

9. Exception translation mechanism
Hibernate translates checked SQL excs --un checked hibernate excs (org.hibernate.HibernateException) : so that prog is not forced to handle the same.

& many more advantages...

Hibernate architecture
refer to a diagram
day6-data\day6_help\hibernate-help\diags\hibernate overview.png

Important Blocks
1.org.hibernate.Session : interface (imple classes : hibernate core jar)
Represents : Main runtime i/f for prog interaction with hibernate
Supplies CRUD APIs(eg : save, persist, get,load, createQuery,update,delete....)
Represents : wrapper around pooled out db connection.
It has INHERENTLY L1 cache(persistence ctx) associated with it.
DAO layer creates session instance as per demand(one per request for CRUD operation)
light weight , thread un safe object
NO NEED for accessing the session in synchronized manner : since different thrd representing different clnt requests , will have their own session object

2. org.hibernate.SessionFactory : interface (imple classes : hibernate core jar)
JOB : to provide session objects(openSession / getCurrentSession)
singleton instance per DB / application
immutable , inherently thread safe
Represents : DB sepecific config , including connection pool.
It has L2 cache associated with it : MUST be configured explcitly

3. Configuration : org.hibernate.cfg.Configuration class.
Provider of SF

4. Additional APIs
, Transaction,Query,CriteriaQuery ...

5. hibernate.cfg.xml : centralized configuration file , to create SessionFactory(i.e bootstrapping hibernate framework)

Important property :
hibernate.hbm2ddl.auto=update
Hibernate checks if table is not yet created for a POJO : create a new table.
BUT if table alrdy exists : continues with the existing table.

5.5 HibernateUtils --- to create singleton immutable SF instance

6. Refer to E-R of Blogs management system
6.1 users table
column - id , first name , last name, email ,password , dob , registration amount,role,image

We are going to confirm auto table creation
POJO ---> Table approach
Create POJO class : User

POJO Annotations

Package : javax.persistence
@Entity : Mandatory : cls level
@Id : Mandatory : field level or property (getter) : PK

Optional annotation for further customization :

@Table(name="tbl_name) : to specify table name n more

@GeneratedValue : to tell hibernate to auto generate ids
auto / identity(auto incr : Mysql) / table / sequence(oracle)
eg : @Id => PK
@GeneratedValue(strategy=GenarationType.IDENTITY) => auto increment

@Column(name,unique,nullable,insertable,updatable,length,columnDefinition="double(8,2)") : for specifying col details

@Transient : Skipped from persistence(no col will be generated in DB table)

@Temporal : java.util.Date , Calendar , GregorianCalendar

LocalDate(date) ,LocalTime(time) , LocalDateTime (timestamp/datetime) : no temporal anno.

@Lob : BLOB(byte[]) n CLOB(char[]) : saving / restoring large bin /char data to/from DB

@Enumerated (EnumType.STRING): enum (def : ordinal : int)

Add <mapping class="F.Q POJO class name"/> in hibernate.cfg.xml

7. Create DAO i/f & write its implementation class
Hibernate based DAO impl class

7.1 No data members ,constructor , cleanup
7.2 Directly add CRUD methods.

Steps in CRUD methods
1. Get hib session from SF
API of org.hibernate.SessionFactory
public Session openSession() throws HibernateException
OR

public Session getCurrentSession() throws HibernateException

2. Begin a Transaction
API of Session
public Transaction beginTransaction()throws HibernateException

3. try {
perform CRUD using Session API (eg : save/get/persist/update/delete/JPQL...)
commit the tx.
} catch(RuntimeException e)
{
roll back tx.
re throw the exc to caller
} finally {
close session --destroys L1 cache , pooled out db cn rets to the pool.
}

4 Refer to Hibernate Session API
(hibernate api-docs & readme : hibernate session api)

5. Create main(..) based Tester & test the application.

Which of the following layers are currently hibernate specific(native hibernate) ?

DAO : org.hibernate.SF , Session, Transaction,Query... : hibernate specific
POJO : javax.persistence : annotations => hibernate inde. (JPA compliant)
Utils : Configuration , org.hibernate.SF => hibernate specific

6. Add a breakpoint before commit , observe n conclude.

7. Replace openSession by getCurrentSession

8. Objective : Get user details
I/P : user id
O/P : User details or error
API : session.get

9. Confirm L1 cache
by invoking session.get(...) multiple times.

10. Hibernate POJO states :
transient , persistent , detached.

11. Objective : Display all user details
Can you solve it using session.get ? :
sql : "select * from users_tbl"
hql : "from User"
jpql : "select u from User u"
u : POJO alias

11.1 Solve it using HQL(Hibernate query language)/JPQL (Java Persistence Query Language)

Object oriented query language, where table names are replaced by POJO class names & column names are replaced by POJO property names, in case sensitive manner.

11.2. Create Query Object --- from Session i/f
<T> org.hibernate.query.Query<T> createQuery(String jpql/hql,Class<T> resultType)
T --result type.

11.3. To execute query
Query i/f method
public List<T> getResultList() throws HibernateException
--Rets list of PERSISTENT entities.

12. Objective : Display all users registered between strt date n end date & under a specific role
I/P : begin dt , end date , role
eg : sql = select * from users where reg_date between ? and ? and user_role=?
jpql =

Passing named IN params to the query
Query i/f method
public Query<T> setParameter(String paramName,Object value) throws HibernateException.

13. User Login (Lab work)
i/p : email n password
o/p User details with success mesg or invalid login mesg

14. Objective : Display all user names registered between strt date n end date & under a specific role
jpql =

15 Objective : Display all user name,reg amount,reg date registered between strt date n end date & under a specific role

String jpql ="select u.name,u.regAmount,u.regDate from User u where u.regDate between :strt and :end and userRole=:rl"

List<Object[]> list=session.createQuery(jpql,Object[].class).setParameter("start", startDate).setParameter("end", endDate).setParameter("rl", userRole).getResultList();
In Tester :
list.forEach(o -> sop(o[0]+" "+o[1]+" "+o[2]));

INSTEAD use a constructor expression
eg :
jpql = "select new pojos.User(name,regAmount,regDate) from User u where u.regDate between :strt and :end and userRole=:rl";

Pre requisite : MATCHING constr in POJO class

17. Update
Objective :
1. Change password
i/p --email , old password , new pass

o/p : mesg indicating success or a failure

2. Apply discount to reg amount , for all users , reged before a specific date.(Bulk update)
i/p -- discount amt, reg date
String jpql="update User u set u.regAmount=u.regAmount-:disc where u.regDate < :dt";

2.1 Query API
public int executeUpdate() throws HibernateException
--use case --DML
2.2 Session API
public Query<T> createQuery(String jpql) throws HibernateException
jpql -- DML

18. Un subscribe user
i/p user id
o/p user details removed from DB
Hint : Session API :

19. Lab work
Objective --delete vendor details for those vendors reg date > dt.
via Bulk delete
String jpql="delete from Vendor v where v.regDate > :dt";

20. Save n restore images to / from DB
FileUtils from Apache commons-io
<!-- https://mvnrepository.com/artifact/commons-io/commons-io -->
```
  <dependency>
    <groupId>commons-io</groupId>
    <artifactId>commons-io</artifactId>
    <version>2.11.0</version>
  </dependency>
```
Methods

1. public static byte[] readFileToByteArray(File file)
                                throws IOException
Reads the contents of a file into a byte array. The file is always closed.

2. public static void writeByteArrayToFile(File file,
                        byte[] data)
                                throws IOException
Writes a byte array to a file creating the file if it does not exist.